ISEN
ALL IS DIGITAL!
OUEST

TP Régression pénalisée

Exo 1: Ridge et lasso avec `scikit-learn`

On considère le jeu de données `ozone.txt` où on cherche à expliquer la concentration maximale en ozone relevée sur une journée (variable `maxO3`) par d'autres variables essentiellement météorologiques.

```
import pandas as pd
import numpy as np

# Chargement des donnees
ozone = pd.read_csv("data/ozone.txt", sep="\s+")
print(ozone.head())
```

Contrairement à R, la librairie `scikit-learn` en Python requiert de séparer explicitement la matrice des X et le vecteur des Y, et de gérer soi-même l'encodage des variables qualitatives (variables catégorielles). On peut obtenir la matrice des X et le codage (dummy variables) avec la fonction `pd.get_dummies` :

```
# On separe la variable cible des variables explicatives
y = ozone['maxO3']
# On binarise les variables qualitatives et on enleve la premiere modalite
X = pd.get_dummies(ozone.drop('maxO3', axis=1), drop_first=True)
```

1. Importer les modèles `Ridge` et `Lasso` depuis `sklearn.linear_model`. À l'aide de ces classes, calculer les estimateurs Ridge et Lasso. On calculera les coefficients pour une grille de paramètres de régularisation `alpha`.

2. Analyser les sorties et plus particulièrement les attributs contenant les coefficients (`coef_`) obtenus en fonction du paramètre `alpha`.

3. Visualiser les chemins de régularisation des estimateurs Ridge et Lasso. On pourra utiliser `matplotlib.pyplot` pour tracer l'évolution des valeurs des coefficients en fonction de `alpha` (ou de $\log(\alpha)$).

4. Sélectionner les paramètres de régularisation optimaux à l'aide des fonctions `RidgeCV` et `LassoCV`. On pourra notamment faire un graphique de l'erreur quadratique moyenne en fonction du paramètre `alpha` et expliquer le graphe obtenu.

5. On souhaite prédire la variable cible pour de nouveaux individus, par exemple les 25ème et 50ème individus du jeu de données (`X.iloc[[24, 49]]`). Calculer les valeurs prédites pour ces deux individus.

6. À l'aide d'une validation croisée (ex: `cross_val_score`), comparer les performances des estimateurs MCO (`LinearRegression`), Ridge et Lasso. On pourra utiliser les données `ozone_complet.txt` qui contiennent plus d'individus et de variables.

```
# Chargement du jeu de donnees complet et suppression des valeurs manquantes
ozone1 = pd.read_csv("data/ozone_complet.txt", sep=";").dropna()

# Preparation des donnees
y1 = ozone1['maxO3']
X1 = pd.get_dummies(ozone1.drop('maxO3', axis=1), drop_first=True)
```

7. Refaire la question précédente en considérant toutes les interactions d'ordre 2 (on pourra utiliser `PolynomialFeatures(degree=2, interaction_only=True)` du module `sklearn.preprocessing`).

Exo 2: Reconstruction d'un signal

Le fichier `signal.csv` contient un signal que l'on peut représenter par une fonction $m : \mathbb{R} \to \mathbb{R}$. On le visualise de la façon suivante :

```
import pandas as pd
import matplotlib.pyplot as plt

signal = pd.read_csv("data/signal.csv")
plt.plot(signal['x'], signal['y'])
plt.title("Signal original")
plt.show()
```

Plaçons-nous dans le cas où on ne dispose que d'une version bruitée de ce signal. La courbe n'est pas observée mais on dispose d'un échantillon (x_i, y_i), pour $i = 1, \dots, n$, généré selon le modèle :

$$y_i = m(x_i) + \epsilon_i$$

Le fichier `ech_signal.csv` contient $n = 60$ observations issues de ce modèle. On représente les données et la courbe ensemble :

```
donnees = pd.read_csv("data/ech_signal.csv")
plt.plot(signal['x'], signal['y'], label="Vrai signal")
# Attention aux noms des colonnes dans ech_signal.csv (ex: 'X' et 'Y')
plt.scatter(donnees['X'], donnees['Y'], color='red', label="Observations")
plt.legend()
plt.show()
```

Nous cherchons dans cette partie à reconstruire le signal à partir de l'échantillon. Bien entendu, vu la forme du signal, un modèle linéaire de la forme :

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i$$

n'est pas approprié. De nombreuses approches en traitement du signal proposent d'utiliser une base ou dictionnaire représentée par une collection de fonctions $\{\psi_j(x)\}_{j=1,\dots,K}$ et de décomposer le signal dans cette base :

$$m(x) \approx \sum_{j=1}^{K} \beta_j \psi_j(x)$$

Pour un dictionnaire donné, on peut alors considérer le modèle linéaire :

$$y_i = \sum_{j=1}^{K} \beta_j \psi_j(x_i) + \epsilon_i$$

Le problème est toujours d'estimer les paramètres β_j, mais les variables sont maintenant définies par les éléments du dictionnaire. Il existe différents types de dictionnaires, dans cet exercice nous proposons de considérer la **base de Fourier** définie par :

$$\psi_0(x) = 1, \quad \psi_{2j-1}(x) = \cos(2j\pi x) \quad \text{et} \quad \psi_{2j}(x) = \sin(2j\pi x), \quad \text{pour } j = 1, \dots, K.$$

1. Écrire une fonction Python (utilisant `numpy`) qui admet en entrée :

   - une grille de valeurs de `x` (un tableau numpy ou une liste)

- une valeur de `K` (un entier positif)

et qui renvoie en sortie une matrice contenant les valeurs du dictionnaire pour chaque valeur de x. Cette matrice de design devra donc contenir $2K + 1$ colonnes (en incluant l'ordonnée à l'origine ψ_0) et le nombre de lignes sera égal à la longueur du vecteur x.

2. On fixe $K = 25$. À l'aide de la classe `LinearRegression`, calculer les estimateurs des moindres carrés du modèle sur l'échantillon bruité.

3. Représenter le signal estimé (superposé aux observations et au vrai signal). Commenter le graphe obtenu.

4. Calculer les estimateurs Lasso (on pourra utiliser `LassoCV` pour trouver un bon niveau de pénalisation) et représenter le signal issu de ces estimateurs.

5. Identifier les coefficients Lasso sélectionnés qui ne sont pas nuls.